# American Option Pricing via Longstaff–Schwartz Monte Carlo

Theory, Implementation, and Numerical Analysis

Kapil

**Abstract**

We implement the Longstaff–Schwartz least-squares Monte Carlo (LSM) method for pricing American options. The algorithm approximates continuation values using regression on simulated paths, enabling efficient valuation of early-exercise derivatives. We analyze convergence, basis-function sensitivity, and numerical stability, and compare results with benchmark methods. The study demonstrates the effectiveness and limitations of LSM for high-dimensional optimal stopping problems.

## 1 Introduction

Pricing American options is fundamentally an optimal stopping problem. Unlike European options, they can be exercised at any time prior to maturity, making valuation significantly more complex. Traditional lattice or PDE methods become computationally expensive in high dimensions, motivating Monte Carlo approaches. However, standard Monte Carlo methods struggle with early exercise decisions.

The Longstaff–Schwartz method overcomes this by estimating continuation values through regression. This project implements the algorithm from scratch and analyzes its numerical behavior.

## 2 Mathematical Formulation

### 2.1 Risk-Neutral Pricing

The value of an American option is

[ $\mathrm{V}_0 = \sup_{\tau \in \mathcal{T}} \mathbb{E}^{\mathbb{Q}} \left[ e^{-r\tau} h(S_\tau) \right]$ , ]

where $\tau$ is a stopping time.

### 2.2 Dynamic Programming Principle

[ $\mathrm{V}_t = \max\left(h_t, ; C_t\right)$ ]

where continuation value

[ $\mathrm{C}_t = \mathbb{E}^{\mathbb{Q}} \left[ e^{-r\Delta t} V_{t+1} \mid \mathcal{F}_t \right]$ .]

### 2.3 Regression Approximation

LSM approximates continuation value as

[ $\mathrm{C}_t(S) \approx \sum_{k=1}^{K} \beta_k \phi_k(S)$.]

## 3 Algorithm Description

### 3.1 Overview

1. **Simulate asset paths under risk-neutral dynamics.**

## 3.2 Simulation

**Risk-Neutral Simulation Path Derivation**

**Step 1 — Start with SDE**

$$dS_t = rS_t dt + \sigma S_t dW_t$$

Divide by $S_t$:

$$\frac{dS_t}{S_t} = r\,dt + \sigma\,dW_t$$

**Step 2 — Apply Itô to** $\ln S_t$ Let

$$X_t = \ln S_t$$

Itô's lemma:

$$df = f'(S)dS + \tfrac{1}{2}f''(S)(dS)^2$$

Derivatives:

$$f'(S) = \frac{1}{S}, \quad f''(S) = -\frac{1}{S^2}$$

First term:

$$\frac{1}{S}(rSdt + \sigma SdW) = rdt + \sigma dW$$

Compute $(dS)^2$.

Since

$$dS = rSdt + \sigma SdW$$

Squaring,

$$(dS)^2 = (rSdt)^2 + (\sigma SdW)^2 + 2(rSdt)(\sigma SdW)$$

Using stochastic rules

$$dt^2 = 0, \quad dt\,dW = 0, \quad (dW)^2 = dt$$

$$(dS)^2 = \sigma^2 S^2 dt$$

Second term:

$$\tfrac{1}{2}\left(-\frac{1}{S^2}\right)\sigma^2 S^2 dt = -\tfrac{1}{2}\sigma^2 dt$$

Combine:

$$d(\ln S_t) = rdt + \sigma dW_t - \tfrac{1}{2}\sigma^2 dt$$

$$d(\ln S_t) = \left(r - \tfrac{1}{2}\sigma^2\right)dt + \sigma dW_t$$

**Step 3 — Integrate**

$$\int_t^{t+\Delta t} d(\ln S_u) = \int_t^{t+\Delta t} \left(r - \tfrac{1}{2}\sigma^2\right) du + \int_t^{t+\Delta t} \sigma dW_u$$

Left side:

$$\ln S_{t+\Delta t} - \ln S_t$$

Right side:

$$\left(r - \tfrac{1}{2}\sigma^2\right) \Delta t + \sigma(W_{t+\Delta t} - W_t)$$

**Step 4 — Brownian Increment**

$$W_{t+\Delta t} - W_t = \sqrt{\Delta t}\, Z, \qquad Z \sim \mathcal{N}(0,1)$$

Substitute:

$$\ln S_{t+\Delta t} - \ln S_t = \left(r - \tfrac{1}{2}\sigma^2\right) \Delta t + \sigma\sqrt{\Delta t}\, Z$$

**Step 5 — Solve**

$$\ln S_{t+\Delta t} = \ln S_t + \left(r - \tfrac{1}{2}\sigma^2\right) \Delta t + \sigma\sqrt{\Delta t}\, Z$$

Exponentiating:

$$S_{t+\Delta t} = S_t \exp\left[\left(r - \tfrac{1}{2}\sigma^2\right) \Delta t + \sigma\sqrt{\Delta t}\, Z\right]$$

**Final Result (Discrete Simulation Formula)**

$$S_{k+1} = S_k \exp\left[\left(r - \tfrac{1}{2}\sigma^2\right) \Delta t + \sigma\sqrt{\Delta t}\, Z_k\right]$$

2. **Compute terminal payoffs.**

   A set of asset price paths is first simulated under risk-neutral dynamics using the exact discretization scheme. The terminal payoff is then computed for each simulated path.

3. **Move backward in time:**

   - Select in-the-money paths
   - Regress discounted future cash flows
   - Estimate continuation values
   - Compare with exercise value
   - Update cashflows

**Regression of Discounted Future Cashflows** We work at a fixed time step $t_k$ (not the final time). Assume we simulated $M$ paths. For each path $m = 1, 2, \ldots, M$ we already know the value of the option at the next time step,

$$V(t_{k+1}, m),$$

which incorporates optimal decisions in the future.

**Step 1 Discount future value to current time**

We bring that value back to time $t_k$:

$$Y_m = e^{-r\Delta t} V(t_{k+1}, m).$$

Thus $Y_m$ represents the value today of the future payoff if we continue.

**Step 2 Target quantity**

We want the continuation value

$$C(S_{t_k}) = \mathbb{E}[\, Y \mid S_{t_k} \,],$$

i.e., the expected future payoff given the current asset price. This conditional expectation is not known analytically and must therefore be approximated.

**Step 3 Regression approximation**

We assume the continuation value can be approximated by basis functions:

$$C(S) \approx \sum_{j=1}^{K} \beta_j \phi_j(S),$$

where $\{\phi_j\}_{j=1}^{K}$ are chosen basis functions (e.g. $1, S, S^2$ or Laguerre polynomials) and $\beta_1, \ldots, \beta_K$ are unknown coefficients.

**Step 4 Path selection**

We use only in-the-money paths:

$$h(S_{t_k}^{(m)}) > 0.$$

Let

$$\mathcal{I}_k = \{m : h(S_{t_k}^{(m)}) > 0\}$$

denote the index set of such paths.

**Step 5 Least-squares problem**

The coefficients are chosen to minimize squared error:

$$\min_{\beta} \sum_{m \in \mathcal{I}_k} \left( Y_m - \sum_{j=1}^{K} \beta_j \phi_j(S_{t_k}^{(m)}) \right)^2 .$$

**Step 6 Estimated continuation value**

After solving this regression, we obtain

$$\widehat{C}(S_{t_k}) = \sum_{j=1}^{K} \beta_j \phi_j(S_{t_k}),$$

which serves as an estimate of the continuation value.

**Step 7 Optimal decision rule**

For each path $m$,

$$V(t_k, m) = \max\left(h(S_{t_k}^{(m)}), \widehat{C}(S_{t_k}^{(m)})\right).$$

Thus, if immediate payoff exceeds continuation value we exercise; otherwise we continue.

4. **Average discounted cashflows.**

**Final Price Estimator** After all optimal stopping decisions have been made and values have been propagated backward to time $t_0$, each simulated path $m = 1, \ldots, M$ yields a time-zero value

$$V(t_0, m).$$

The Monte Carlo estimator of the American option price is the cross-sectional average:

$$\widehat{V}_0 = \frac{1}{M} \sum_{m=1}^{M} V(t_0, m).$$

This estimator is consistent as $M \to \infty$ and represents the discounted expected payoff under the optimal stopping strategy.

### 3.3 Pseudocode

```
Simulate paths
Initialize cashflows at maturity

for t = T–1 to 1:
select ITM paths
regress future payoff on basis
compute continuation value
exercise if payoff > continuation
update cashflows

return discounted mean cashflow
```

## 4 Implementation Details

This section describes the practical implementation of the Longstaff–Schwartz Monte Carlo (LSM) algorithm in Python, including market calibration, path simulation, regression methodology, and numerical considerations.

### 4.1 Market Data Calibration

Historical market data for Apple Inc. (AAPL) was downloaded using the `yfinance` API. The most recent adjusted closing price was used as the initial asset price $S_0$.

Volatility was estimated from daily percentage returns:

$$\sigma = \mathrm{std}(R_t)\sqrt{252},$$

where 252 denotes the average number of trading days in a year. This produces an annualized volatility estimate consistent with market convention.

The model parameters used in the simulation were:

$$r = 0.05, \quad T = 1 \text{ year}, \quad N = 50 \text{ time steps},$$

with

$$\Delta t = \frac{T}{N}.$$

A total of $M = 1000$ Monte Carlo paths were simulated.

### 4.2 Path Simulation

Asset price paths were simulated under the risk-neutral geometric Brownian motion using the exact discretization:

$$S_{t+\Delta t} = S_t \exp\left[\left(r - \tfrac{1}{2}\sigma^2\right)\Delta t + \sigma\sqrt{\Delta t}\, Z\right], \quad Z \sim \mathcal{N}(0,1).$$

The simulated paths were stored in a matrix of dimension

$$M \times (N+1),$$

where each row represents one simulated trajectory and each column corresponds to a time step.

Independent Gaussian random variables were generated at each time step using NumPy's random number generator. The overall computational complexity of path generation is

$$O(MN),$$

which scales linearly in both the number of paths and the number of time steps.

### 4.3 LSM Backward Induction and Regression

An American put option with strike $K = S_0$ was considered. The terminal payoff was initialized as

$$V_T^{(m)} = \max(K - S_T^{(m)}, 0).$$

Backward induction was then applied from $t_{N-1}$ down to $t_1$.

At each time step:

1. **In-the-money filtering.**

   Paths satisfying

   $$K - S_t^{(m)} > 0$$

   were selected for regression. Restricting to in-the-money paths improves stability and reduces noise in continuation value estimation.

2. **Discounted future payoff.**

   For selected paths, the discounted continuation target was computed:

$$Y^{(m)} = e^{-r\Delta t} V_{t+1}^{(m)}.$$

3. **Regression approximation.**

   The continuation value was approximated using a polynomial basis:

$$\{1, S_t, S_t^2\}.$$

   The least-squares problem solved at each step was:

$$\min_{\beta} \sum_{m \in \mathcal{I}_t} \left( Y^{(m)} - \beta_0 - \beta_1 S_t^{(m)} - \beta_2 S_t^{(m)2} \right)^2.$$

   In matrix form,

$$\min_{\beta} \|A\beta - Y\|^2,$$

   where

$$A = \begin{bmatrix} 1 & S & S^2 \end{bmatrix}.$$

   The coefficients were computed using NumPy's `lstsq` routine, which internally applies QR decomposition. This approach avoids direct inversion of $A^\top A$ and improves numerical stability.

4. **Optimal exercise decision.**

   For each path,

$$V_t^{(m)} = \max\left(K - S_t^{(m)}, \widehat{C}(S_t^{(m)})\right).$$

   If immediate exercise exceeded continuation value, the option was exercised; otherwise, the discounted continuation value was retained.

## 4.4 Final Price Estimation

After backward induction reached $t_0$, the American option price was estimated as

$$\widehat{V}_0 = e^{-r\Delta t} \frac{1}{M} \sum_{m=1}^{M} V_0^{(m)}.$$

This Monte Carlo estimator is consistent as $M \to \infty$.

## 4.5 Numerical Considerations

Several important numerical aspects arise in the implementation:

- **Monte Carlo variance.** Using $M = 1000$ paths provides a reasonable estimate but introduces noticeable statistical noise. Increasing $M$ reduces variance at rate $O(M^{-1/2})$.
- **Basis selection bias.** A low-degree polynomial basis reduces overfitting but may introduce approximation bias if the true continuation function is more complex.

- **Ill-conditioning.** High-degree polynomial bases may lead to near-singular design matrices. Using QR-based least squares improves numerical stability.
- **In-the-money path scarcity.** When few paths are in the money, the regression becomes unstable due to insufficient data points. This is a known limitation of LSM in deep out-of-the-money regions.
- **Bias of LSM estimator.** The LSM algorithm produces a downward-biased estimator because the stopping rule is suboptimal when the continuation value is approximated.
- **Computational complexity.** The overall complexity is approximately linear in both the number of paths and time steps:

$$O(MN).$$

# 5 References


1. Longstaff, F. A., Schwartz, E. S. (2001). Valuing American Options by Simulation.
2. Hilpisch, Y. (2015). *Derivatives Analytics with Python: Data Analysis, Models, Simulation, Calibration and Hedging.* Wiley Finance.